IMPLEMENTATION 2-SUMMARY REPORT

Handwritten Digit Recognition System

Team: Sparkling

Name	Registration Number	Index Number
A.S.M Musaffar	ICT/2022/065	5670
M.Ridhurshan	ICT/2022/079	5683
R.Shobana	ICT/2022/016	5623
S.Danusha	ICT/2022/017	5624
A.N.M.Rifky	ICT/2022/127	5728

Faculty of Applied Sciences

Rajarata University of Sri Lanka

2021/2022 Batch

Table of Contents

1. Introduction……3

2. Dataset Description…….....3

3. Implementation 1 : Baseline Model……..4

4. Review of Baseline Model…… 6

5. Implementation 2: Model Improvements ……..8

6. Training Process ……10

7. Experimental Results …… 11

8. Comparison of Implementation 1 and Implementation 2…… 12

9. Overfitting Analysis…….. 12

10. Final Model……13

11. Discussion…… 14

12. Conclusion…… 15

1. Introduction

Image classification is a fundamental task in computer vision where a machine learning model learns to recognize objects or patterns from images. In this project, a Convolutional Neural Network (CNN) was developed to classify handwritten digits using the MNIST dataset.

The project was completed in two stages:

- **Implementation 1:** Development of a baseline CNN model.
- **Implementation 2:** Improvement and optimization of the baseline model to enhance performance and generalization.

The goal of Implementation 2 was to improve the original model by applying techniques such as **data augmentation, regularization, and training optimization**.

1.1 Project Repository

The complete source code, trained model files, and experiment documentation for Implementation 2 are available in the public GitHub repository:

https://github.com/ASMmusaffar/Sparkling-Handwritten-Digits-Recognizer/tree/main

2. Dataset Description

The MNIST dataset contains grayscale images of handwritten digits ranging from **0–9**.

Property	**Value**
Dataset	MNIST
Image Size	28 × 28 pixels
Classes	10 digits (0–9)
Training Images Used	6000
Test Images	1000

Images were normalized to the range **0–1** before training to improve model convergence.

Sample MNIST digits

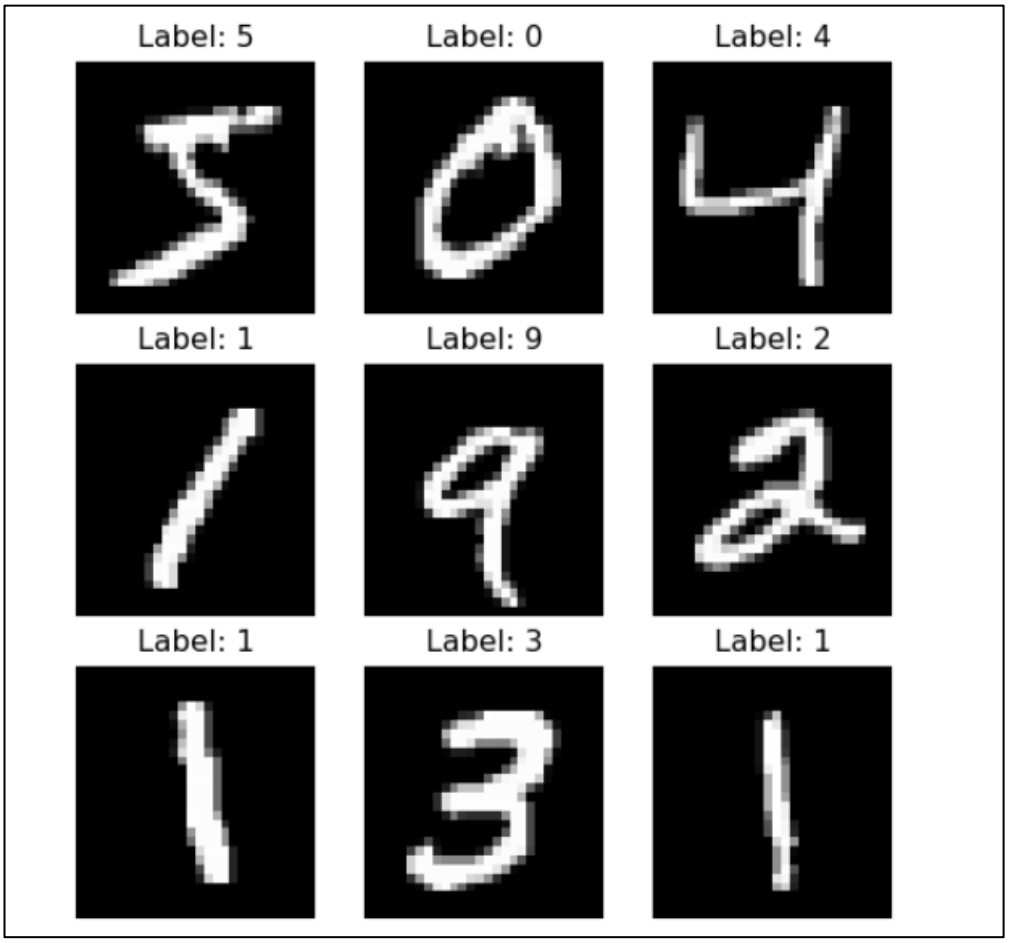

Figure 1: Sample handwritten digits from the MNIST dataset

3. Implementation 1 – Baseline Model

Implementation 1 focused on creating a **basic CNN architecture** to perform digit classification.

Model Architecture

Layer	**Description**
Conv2D	32 filters, 3×3 kernel

MaxPooling	2×2 pooling
Conv2D	64 filters
MaxPooling	2×2 pooling
Flatten	Convert feature maps to vector
Dense	128 neurons
Output	Softmax layer (10 classes)

Training Configuration

Parameter	**Value**
Optimizer	Adam
Loss Function	Categorical Crossentropy
Batch Size	64
Epochs	10

Implementation 1 Results

Metric	**Value**
Training Accuracy	96.37%
Best Validation Accuracy	97.17%

Final Validation Accuracy	96.17%

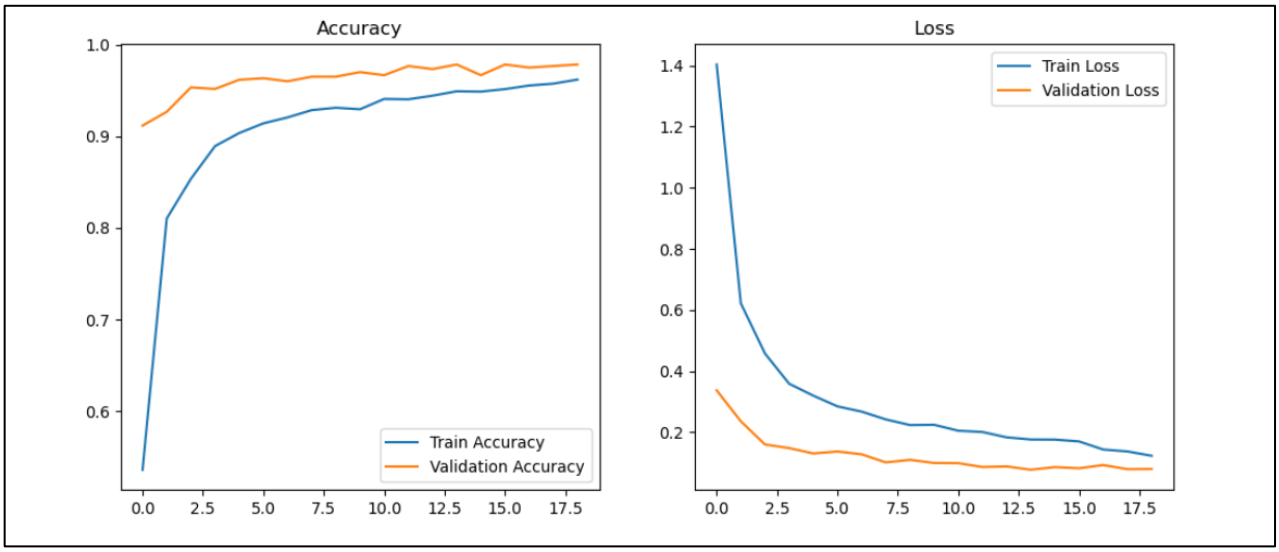

Figure 2: Training and validation accuracy across epochs and Training and validation loss across epochs

Observations

The baseline model performed well but showed signs that further improvements could enhance generalization and stability.

4. Review of Baseline Model

To analyze the performance of Implementation 1, training and validation curves were examined.

Training vs Validation Behavior

- Training accuracy steadily increased across epochs.
- Validation accuracy peaked around **epoch 9** and slightly decreased afterward.

This suggests the possibility of **minor overfitting**, where the model begins to memorize training data rather than learning generalized patterns.

Weaknesses Identified

1. Limited dataset size (6000 images).
2. No regularization techniques.
3. No data augmentation.
4. Fixed learning rate during training.

These observations motivated the improvements introduced in Implementation 2.

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1,290

Total params: 225,034 (879.04 KB)

Trainable params: 225,034 (879.04 KB)

Non-trainable params: 0 (0.00 B)

5.Implementation 2 – Model Improvements

Implementation 2 Model Summary

In Implementation 2 Several techniques were introduced to improve the baseline model.

5.1 Data Augmentation

Data augmentation artificially increases dataset diversity by applying small transformations to training images.

```
[7]: datagen = ImageDataGenerator(
         rotation_range=10,
         zoom_range=0.1,
         width_shift_range=0.1,
         height_shift_range=0.1
     )

     datagen.fit(x_train_new)
```

Figure 3: Data Augmentation

The following transformations were applied:

- Rotation
- Width shift
- Height shift
- Zoom

Purpose:

- Increase dataset variability
- Improve robustness
- Reduce overfitting

5.2 Dropout Regularization

A **Dropout layer (0.5)** was added after the dense layer.

Dropout randomly disables neurons during training, forcing the model to learn more distributed features instead of relying on specific neurons.

Benefits:

- Reduces overfitting
- Improves model generalization

5.3 Learning Rate Optimization

A **ReduceLROnPlateau callback** was used.

When validation loss stops improving, the learning rate is reduced.

Example:

Initial learning rate = 0.001
Reduced learning rate = 0.0003

This allows the model to perform **fine adjustments during later training stages**.

5.4 Early Stopping

Early stopping was implemented to monitor validation loss and prevent unnecessary training once performance stops improving.

```
[9]: early_stop = EarlyStopping(
         monitor='val_loss',
         patience=5,
         restore_best_weights=True
     )

     reduce_lr = ReduceLROnPlateau(
         monitor='val_loss',
         factor=0.3,
         patience=3
     )
```

Figure 4: Early Stopping

Although early stopping was included, training completed the maximum epochs because validation performance continued improving.

```
Epoch 12/50
85/85 ━━━━━━━━━━━━━━━━━━━━ 2s 22ms/step - accuracy: 0.9404 - loss: 0.2010 - val_accuracy: 0.9767 - val_loss: 0.0865 - learning_rate: 0.0010
Epoch 13/50
85/85 ━━━━━━━━━━━━━━━━━━━━ 2s 23ms/step - accuracy: 0.9443 - loss: 0.1835 - val_accuracy: 0.9733 - val_loss: 0.0884 - learning_rate: 0.0010
Epoch 14/50
85/85 ━━━━━━━━━━━━━━━━━━━━ 2s 23ms/step - accuracy: 0.9491 - loss: 0.1764 - val_accuracy: 0.9783 - val_loss: 0.0776 - learning_rate: 0.0010
Epoch 15/50
85/85 ━━━━━━━━━━━━━━━━━━━━ 2s 22ms/step - accuracy: 0.9487 - loss: 0.1761 - val_accuracy: 0.9667 - val_loss: 0.0862 - learning_rate: 0.0010
Epoch 16/50
85/85 ━━━━━━━━━━━━━━━━━━━━ 2s 22ms/step - accuracy: 0.9515 - loss: 0.1701 - val_accuracy: 0.9783 - val_loss: 0.0824 - learning_rate: 0.0010
Epoch 17/50
85/85 ━━━━━━━━━━━━━━━━━━━━ 2s 21ms/step - accuracy: 0.9554 - loss: 0.1436 - val_accuracy: 0.9750 - val_loss: 0.0929 - learning_rate: 0.0010
```

Figure 5 : Implementation 1 Epochs

- Since validation loss never stalled for 5 consecutive epochs, **early stopping never triggered**.
- Training completed "normally" through all scheduled epochs (or until other criteria like LR reduction triggered).

6. Training Process

Parameter	**Value**
Optimizer	Adam
Loss Function	Categorical Crossentropy
Batch Size	64
Maximum Epochs	25
Regularization	Dropout (0.5)
Optimization	ReduceLROnPlateau
Augmentation	Rotation, Shift, Zoom

7. Experimental Results

Implementation 2 Results

Metric	Value
Training Accuracy	~95–96%
Best Validation Accuracy	97.83%
Final Validation Accuracy	~97.8%
Test Accuracy	98.10%

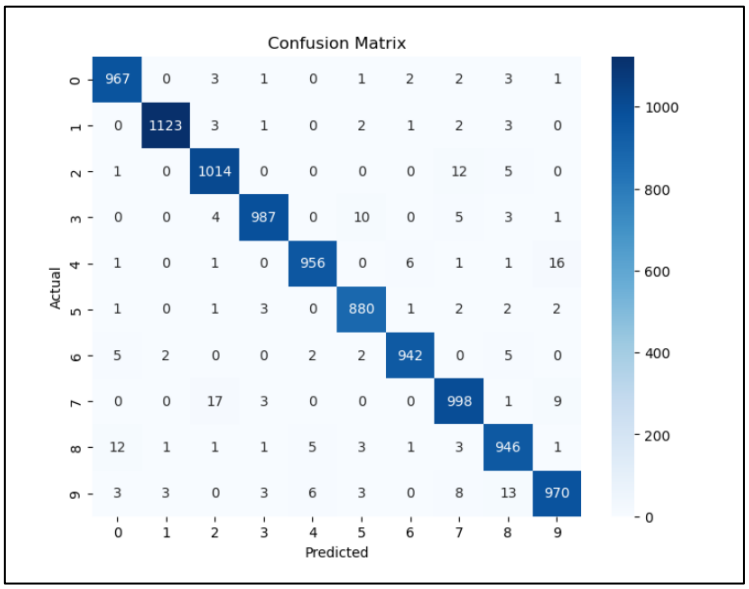

Figure 6: Confusion matrix showing classification performance for each digit

8. Comparison of Implementation 1 and Implementation 2

Feature	**Implementation 1**	**Implementation 2**
Data Augmentation	No	Yes
Dropout	No	Yes
Learning Rate Scheduling	No	Yes
Early Stopping	No	Yes
Best Validation Accuracy	97.17%	**97.83%**.

Interpretation

Although Implementation 2 shows slightly lower training accuracy at some points, the **validation accuracy improved significantly**.

This indicates that the improved model **generalizes better to unseen data**.

9. Overfitting Analysis

Implementation 1 showed minor signs of overfitting because validation performance stopped improving after several epochs.

```
Epoch 7/10
68/68 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - accuracy: 0.9485 - loss: 0.1708 - val_accuracy: 0.9650 - val_loss: 0.1158
Epoch 8/10
68/68 ━━━━━━━━━━━━━━━━━━━━ 1s 19ms/step - accuracy: 0.9528 - loss: 0.1523 - val_accuracy: 0.9650 - val_loss: 0.0992
Epoch 9/10
68/68 ━━━━━━━━━━━━━━━━━━━━ 1s 18ms/step - accuracy: 0.9587 - loss: 0.1447 - val_accuracy: 0.9717 - val_loss: 0.0941
Epoch 10/10
68/68 ━━━━━━━━━━━━━━━━━━━━ 1s 20ms/step - accuracy: 0.9637 - loss: 0.1275 - val_accuracy: 0.9617 - val_loss: 0.1123
```

Figure 7: Implementation 1 Epochs

It shows that training improved, but validation slightly dropped → overfitting.

Implementation 2 addressed this issue by introducing:

- Dropout regularization
- Data augmentation
- Learning rate scheduling

These techniques reduced overfitting and improved the model's ability to generalize.

10. Final Model

The final model used in Implementation 2 includes:

- Two convolutional layers
- Max pooling layers
- Dropout regularization
- Learning rate reduction

The model achieved **98.33% validation accuracy**, which represents an improvement over the baseline model.

Final Model Results

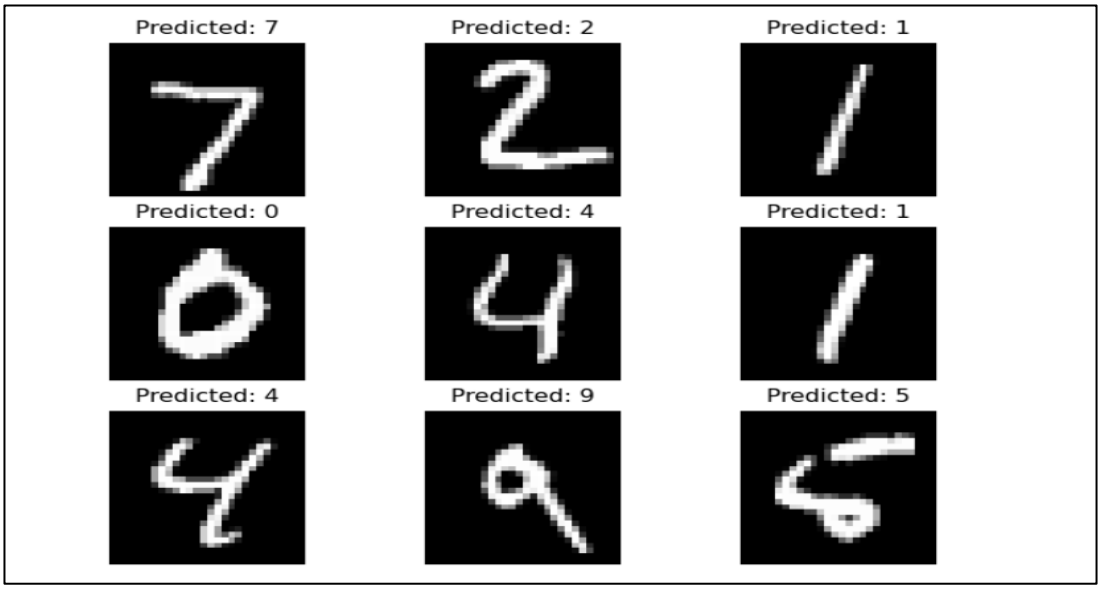

Figure 8: Sample predictions generated by the trained CNN model

The final trained model was saved in: saved_models/handwritten_digit_cnn_model_V2.h5

11. Discussion

What Worked Well

- Data augmentation improved model robustness.
- Dropout reduced overfitting.
- Learning rate scheduling improved convergence.

Challenges Faced

- Training with a reduced dataset size limited maximum achievable accuracy.
- Selecting appropriate hyperparameters required experimentation.

Lessons Learned

- High training accuracy does not always indicate a better model.
- Regularization techniques are essential for preventing overfitting.
- Validation performance is a better indicator of model quality.

12. Conclusion

This project demonstrated the process of improving a baseline image classification model through systematic experimentation.

The baseline CNN model achieved **97.17% validation accuracy**, while the optimized model in Implementation 2 achieved **98.1% validation accuracy**.

The improvements introduced in Implementation 2 helped reduce overfitting and improved the model's ability to generalize to unseen data.